TikuOS MPU Write-Protection Wrappers

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 11, 2026

Outline

1. Why FRAM Write-Protection?
2. MSP430 MPU Hardware
3. Architecture & Types
4. Platform Layering
5. API Reference
6. Usage Examples
7. File Organization
8. API Summary
9. Test Suite
10. Summary

The Problem: Unprotected FRAM

Without MPU protection:

- Wild pointers can corrupt FRAM silently
- Runaway code can overwrite calibration data
- ISR bugs can damage persistent state
- Stack overflows can reach FRAM regions
- Corruption persists across reboots

TikuOS MPU wrappers:

- Default: FRAM is read+execute, **no write**
- Explicit unlock/lock for intentional writes
- Scoped write helper disables interrupts
- Per-segment permission control
- Password-protected hardware registers
- Violation detection via NMI

FRAM Makes It Worse

Unlike SRAM, FRAM is non-volatile — a single wild write persists across power cycles. A corrupted calibration value stays corrupted until explicitly fixed.

Defense in Depth

The MPU acts as a hardware safety net — even if software has a bug, it cannot accidentally modify FRAM without going through the controlled unlock sequence.

MPU Register Architecture

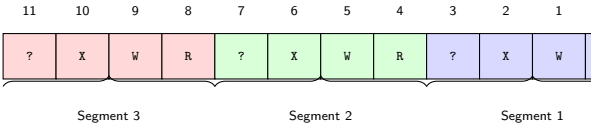
Key registers:

Register	Purpose
`MPUCTL0`	Control: password (upper byte), enable bit, lock bit, SEGIE
`MPUCTL1`	Violation flags (which segment was violated)
`MPUSAM`	Segment Access Mode: R/W/X permissions per segment
`MPUSEGB1/2`	Segment boundary addresses

Password Protection

Every write to `MPUCTL0` must include `0xA500` in the upper byte. Without it, the hardware ignores the write (or triggers a reset). This prevents wild pointers from accidentally changing memory protection.

MPUSAM bit layout:



Each segment has 3 permission bits (R/W/X) in a 4-bit nybble.

Default value `0x0555`**:**
`0x5` = `0b0101` = R+X (no W) per segment.
Three segments: `0x0555`.

Unlock value `0x0222`**:**
ORing this sets the W bit in all three segments.

MPU Types (kernel/memory/tiku_mem.h)

Segment identifiers

```
typedef enum {
    TIKU_MPU_SEG1 = 0,
    TIKU_MPU_SEG2 = 1,
    TIKU_MPU_SEG3 = 2
} tiku_mpu_seg_t;
```

Permission flags

```
typedef enum {
    TIKU_MPU_READ     = 0x01,
    TIKU_MPU_WRITE    = 0x02,
    TIKU_MPU_EXEC     = 0x04,
    TIKU_MPU_RD_WR    = 0x03,
    TIKU_MPU_RD_EXEC  = 0x05,
    TIKU_MPU_ALL      = 0x07
} tiku_mpu_perm_t;
```

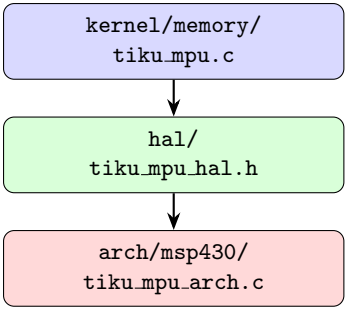
Scoped write callback

```
typedef void (*tiku_mpu_write_fn)(
    void *ctx);
```

Design Choices

- Permission flags map directly to hardware bit positions
- Compound permissions (`RD_WR`, `RD_EXEC`) are pre-defined for convenience
- Callback takes an opaque `void *ctx` so the caller can pass any data without globals

Three-Layer Architecture



Platform-independent logic:
init, set_permissions, unlock, lock, scoped_write, violation

Routes to arch header;
declares HAL function signatures

MSP430 hardware access:
MPUCTL0 password, MPUSAM, boundaries, NMI ISR

Why This Layering?

The kernel MPU code (`tiku_mpu.c`) contains only delegation and sequencing logic — no `#include <msp430.h>`, no register names, no intrinsics. Porting to a new platform means writing a new arch file; the kernel code stays untouched.

HAL Interface

hal/tiku_mpu_hal.h — Low-level

```c
/* Read/write segment-access register */
uint16_t tiku_mpu_arch_get_sam(void);
void tiku_mpu_arch_set_sam(uint16_t sam);

/* Read MPU control register */
uint16_t tiku_mpu_arch_get_ctl(void);

/* Interrupt control */
void tiku_mpu_arch_disable_irq(void);
void tiku_mpu_arch_enable_irq(void);
```

hal/tiku_mpu_hal.h — Higher-level

```c
/* Configure segment boundaries */
void tiku_mpu_arch_init_segments(void);

/* Set default R+X protection */
void tiku_mpu_arch_set_default_protection(void
    );

/* Per-segment permissions */
void tiku_mpu_arch_set_seg_perm(
    uint8_t seg, uint8_t perm);

/* NVM unlock/lock (all segments) */
uint16_t tiku_mpu_arch_unlock_nvm(void);
void tiku_mpu_arch_lock_nvm(
    uint16_t saved_state);

/* Violation detection */
void tiku_mpu_arch_enable_violation_nmi(void);
uint16_t tiku_mpu_arch_get_violation_flags(
    void);
void tiku_mpu_arch_clear_violation_flags(void)
    ;
```

MSP430 Arch Implementation

arch/msp430/tiku_mpu_arch.c

```c
#include "tiku_mpu_arch.h"
#include <msp430.h>

void tiku_mpu_arch_init_segments(void) {
    MPUCTL0  = MPUPW;                                   /* Unlock config */
    MPUSEGB1 = TIKU_DEVICE_MPU_SEG2_START >> 4;     /* Segment 2 boundary */
    MPUSEGB2 = TIKU_DEVICE_MPU_SEG3_START >> 4;     /* Segment 3 boundary */
    MPUCTL0  = MPUPW | MPUENA;                          /* Re-enable MPU */
}

void tiku_mpu_arch_set_sam(uint16_t sam) {
    uint16_t flags = MPUCTL0 & MPUSEGIE;  /* Preserve violation NMI enable */
    MPUCTL0 = MPUPW;                       /* Unlock with password (0xA500) */
    MPUSAM  = sam;                          /* Set new permissions */
    MPUCTL0 = MPUPW | MPUENA | flags;     /* Re-enable + preserved flags */
}

void tiku_mpu_arch_disable_irq(void) { __disable_interrupt(); }
void tiku_mpu_arch_enable_irq(void)  { __enable_interrupt(); }
```

All Hardware Access in One File

The password (MPUPW), enable bit (MPUENA), segment boundaries, NMI configuration, and MSP430

tiku_mpu_init()

```c
void tiku_mpu_init(void);
```

- Configures MPU segment boundaries, then sets all three segments to **read + execute, no write**
- Called by `tiku_mem_init()` at boot — first thing before any other subsystem
- After this call, any attempt to write to FRAM triggers an MPU violation

kernel/memory/tiku_mpu.c

```c
void tiku_mpu_init(void) {
    tiku_mpu_arch_init_segments();          /* Set up segment boundaries */
    tiku_mpu_arch_set_default_protection();  /* R+X on all 3 segments (0x0555) */
}
```

Boot Sequence

`tiku_mem_init()` calls `tiku_mpu_init()` first, then `tiku_mem_arch_init()`. Segment boundaries are configured before protection is applied, and FRAM is write-protected before any other memory subsystem initialization runs.

tiku_mpu_unlock_nvm() / tiku_mpu_lock_nvm()

Unlock: enable writes

```
uint16_t tiku_mpu_unlock_nvm(void)
{
    return tiku_mpu_arch_unlock_nvm();
}
```

Arch implementation

```
uint16_t tiku_mpu_arch_unlock_nvm(void)
{
    uint16_t saved =
        tiku_mpu_arch_get_sam();

    tiku_mpu_arch_set_sam(
        saved | 0x0222);

    return saved;
}
```

- Saves current SAM state
- ORs in write bit for all segments

Lock: restore state

```
void tiku_mpu_lock_nvm(
    uint16_t saved_state)
{
    tiku_mpu_arch_lock_nvm(saved_state);
}
```

Arch implementation

```
void tiku_mpu_arch_lock_nvm(
    uint16_t saved_state)
{
    tiku_mpu_arch_set_sam(saved_state);
}
```

- Restores SAM to pre-unlock value
- Removes write permission
- Safe to call when already locked

tiku_mpu_scoped_write()

```
void tiku_mpu_scoped_write(tiku_mpu_write_fn fn, void *ctx);
```

kernel/memory/tiku_mpu.c

```
void tiku_mpu_scoped_write(tiku_mpu_write_fn fn, void *ctx) {
    uint16_t saved;

    tiku_mpu_arch_disable_irq();   /* 1. Disable interrupts */
    saved = tiku_mpu_unlock_nvm(); /* 2. Unlock NVM */

    fn(ctx);                        /* 3. Execute write function */

    tiku_mpu_lock_nvm(saved);      /* 4. Relock NVM */
    tiku_mpu_arch_enable_irq();    /* 5. Re-enable interrupts */
}
```

Why Disable Interrupts?

While NVM is unlocked, any ISR has write access. A bug in an ISR could corrupt persistent data. Disabling interrupts ensures only the caller's function writes to NVM.

Keep It Short

While interrupts are disabled, hardware events queue up. The write function must be brief — long operations should be split across multiple scoped_write calls.

tiku_mpu_set_permissions()

```
void tiku_mpu_set_permissions(tiku_mpu_seg_t seg, tiku_mpu_perm_t perm);
```

kernel/memory/tiku_mpu.c — delegates to arch

```
void tiku_mpu_set_permissions(tiku_mpu_seg_t seg, tiku_mpu_perm_t perm) {
    tiku_mpu_arch_set_seg_perm((uint8_t)seg, (uint8_t)perm);
}
```

arch/msp430/tiku_mpu_arch.c — register manipulation

```
void tiku_mpu_arch_set_seg_perm(uint8_t seg, uint8_t perm) {
    uint16_t shift = (uint16_t)seg * 4U;
    uint16_t mask  = (uint16_t)0x07 << shift;
    uint16_t sam   = tiku_mpu_arch_get_sam();

    sam = (sam & ~mask) | (((uint16_t)perm & 0x07) << shift);
    tiku_mpu_arch_set_sam(sam);
}
```

- Modifies only the targeted segment's 3-bit permission field
- Other segments are left untouched
- All SAM bit manipulation is in the arch layer — kernel just delegates

Example: Make segment 3 writable while keeping segments 1–2 locked:

Violation Detection

kernel/memory/tiku_mpu.c

```c
void tiku_mpu_enable_violation_nmi(void)
{
    tiku_mpu_arch_enable_violation_nmi();
}

uint16_t tiku_mpu_get_violation_flags(void)
{
    return
      tiku_mpu_arch_get_violation_flags();
}

void tiku_mpu_clear_violation_flags(void)
{
    tiku_mpu_arch_clear_violation_flags();
}
```

arch/msp430/tiku_mpu_arch.c

```c
static volatile uint16_t
    latched_violation_flags;

void tiku_mpu_arch_enable_violation_nmi(void)
{
    MPUCTL0 = MPUPW | MPUENA | MPUSEGIE;
}

/* SYSNMI ISR --- latch flags before
   SYSSNIV read clears them */
TIKU_ISR(SYSNMI_VECTOR,
         tiku_mpu_sysnmi_isr)
{
    latched_violation_flags |= MPUCTL1;
    (void)SYSSNIV; /* acknowledge NMI */
}
```

Usage Pattern

```c
tiku_mpu_enable_violation_nmi();
tiku_mpu_clear_violation_flags();
//      code that may violate
```

Why NMI Instead of Reset?

By default, an MPU violation causes a device reset, losing all state. Setting `MPUSEGIE` switches

When to Use MPU Protection

MPU Protection IS Right When...

- Calibration data in FRAM must survive bugs
- ISRs run alongside FRAM-writing code
- Persistent state must not be silently corrupted
- You want hardware-enforced access control
- You need to diagnose wild-pointer bugs

MPU Protection Is NOT Needed When...

- All data lives in SRAM (volatile anyway)
- Only one task ever writes to FRAM
- You are prototyping and not yet hardening
- The MCU has no MPU (some MSP430 variants)

Rule of Thumb

If you store anything in FRAM that must be correct after the next reboot — calibration, configuration, boot counter — protect it with the MPU. The cost is a few cycles per write; the benefit is catching bugs that would otherwise cause silent, persistent corruption.

Example 1: The Bug MPU Catches

Without MPU — silent corruption

```c
/* Calibration stored in FRAM */
static uint16_t __attribute__((
    section(".persistent")))
    adc_offset = 42;

/* Bug: uninitialized pointer in ISR */
void __attribute__((interrupt))
TIMER_ISR(void)
{
    uint16_t *p;  /* uninitialised! */
    *p = 0xFFFF;  /* wild write */
    /* If p happens to point at
       adc_offset, calibration is
       silently destroyed.
       Persists across reboots. */
}
```

With MPU — fault caught instantly

```c
/* Same bug, but MPU is enabled */
void __attribute__((interrupt))
TIMER_ISR(void)
{
    uint16_t *p;  /* uninitialised! */
    *p = 0xFFFF;  /* wild write */
    /* MPU detects write to locked
       FRAM segment -> triggers NMI
       instead of silent corruption.
       adc_offset remains intact. */
}
```

Key Insight

Without MPU, the bug is invisible — the device reboots with wrong calibration and you have no idea when or how it happened. With MPU, the NMI fires immediately, giving you the PC value and the violated segment.

Example 2: Scoped Write for Sensor Calibration

```c
/* Context for the scoped write callback */
typedef struct {
    tiku_persist_store_t *store;
    const uint8_t *data;
    tiku_mem_arch_size_t len;
} cal_write_ctx_t;

static void write_cal(void *arg) {
    cal_write_ctx_t *ctx = (cal_write_ctx_t *)arg;
    tiku_persist_write(ctx->store, "cal", ctx->data, ctx->len);
}

void save_calibration(tiku_persist_store_t *store,
                      const uint8_t *cal_data, tiku_mem_arch_size_t len)
{
    cal_write_ctx_t ctx = { store, cal_data, len };

    /* scoped_write: disable IRQ -> unlock NVM -> write -> lock -> enable IRQ */
    tiku_mpu_scoped_write(write_cal, &ctx);
}
```

Safety Guarantees

NVM is writable only during the `write_cal` callback. Interrupts are disabled so no ISR can interfere.

Example 3: Per-Segment Protection Policies

Bootloader vs application FRAM

```
/* Segment 1: bootloader code (FRAM)
     -> read + execute, NEVER write
   Segment 2: application code (FRAM)
     -> read + execute, no write
   Segment 3: persistent data (FRAM)
     -> read + write (controlled) */

void boot_mpu_setup(void) {
    tiku_mpu_init();  /* all R+X */

    /* Seg 1: bootloader is immutable
       (even scoped_write won't help) */
    tiku_mpu_set_permissions(
        TIKU_MPU_SEG1, TIKU_MPU_RD_EXEC);

    /* Seg 3: data segment gets R+W
       so persist_write can work */
    tiku_mpu_set_permissions(
        TIKU_MPU_SEG3, TIKU_MPU_RD_WR);
}
```

Tighten after init completes

```
void app_start(void) {
    /* During init, seg 3 was R+W
       for persist_register/write.
       Now lock it down. */
    tiku_mpu_set_permissions(
        TIKU_MPU_SEG3, TIKU_MPU_READ);

    /* From here on, only scoped_write
       can write to segment 3.
       Segments 1 and 2 remain R+X
       throughout. */
}
```

Why Per-Segment?

A sensor node's bootloader must never be overwritten — even by kernel code. Separating bootloader, application, and data into different segments with independent permissions gives

Example 4: Violation Detection for Debugging

```c
void debug_mpu_setup(void) {
    tiku_mpu_init();
    tiku_mpu_enable_violation_nmi();   /* NMI instead of reset on violation */
    tiku_mpu_clear_violation_flags();
}

/* Call periodically or after suspicious behaviour */
void check_for_violations(void) {
    uint16_t flags = tiku_mpu_get_violation_flags();
    if (flags != 0) {
        /* Which segment was violated?
           Bit 0 = seg 1, bit 1 = seg 2, bit 2 = seg 3 */
        if (flags & 0x01) uart_puts("SEG1 violated (bootloader region!)\r\n");
        if (flags & 0x02) uart_puts("SEG2 violated (app code region)\r\n");
        if (flags & 0x04) uart_puts("SEG3 violated (data region)\r\n");

        tiku_mpu_clear_violation_flags();
    }
}
```

Development vs Production

During development, enable violation NMI and log which segment was hit — this pinpoints wild-pointer bugs immediately. In production, you may prefer the default behaviour (device reset on violation) to

Example 5: Manual Unlock for Batched Writes

```c
void batch_persist_update(tiku_persist_store_t *store) {
    uint16_t saved;
    uint8_t cfg_data[] = {0x01, 0x02};
    uint8_t cal_data[] = {0xAA, 0xBB, 0xCC};

    /* Batch multiple NVM writes in one unlock window */
    saved = tiku_mpu_unlock_nvm();

    tiku_persist_write(store, "cfg", cfg_data, sizeof(cfg_data));
    tiku_persist_write(store, "cal", cal_data, sizeof(cal_data));

    tiku_mpu_lock_nvm(saved);
}
```

Trade-off: Batching vs Safety

Manual unlock/lock avoids per-write overhead but leaves NVM unprotected for longer. Use `scoped_write` for single writes; manual unlock for known-safe batches.

When to Batch

- Writing config + calibration together
- Initial provisioning at factory
- Firmware update writing multiple keys
- Any case where all writes are in one function

Module File Organization

Layer	File	Role
Kernel (portable)	`kernel/memory/tiku_mem.h`	Unified API: arena + persist + MPU types
	`kernel/memory/tiku_mem.c`	Arena allocator, module init
	`kernel/memory/tiku_mpu.c`	MPU logic: init, unlock, lock, scoped write
HAL	`hal/tiku_mem_hal.h`	Routes to arch memory header
	`hal/tiku_mpu_hal.h`	Routes to arch MPU header
Arch (MSP430)	`arch/msp430/tiku_mpu_arch.h`	MPU arch function declarations
	`arch/msp430/tiku_mpu_arch.c`	MPUCTL0/MPUSAM access, boundaries, NMI ISR
Tests	`tests/memory/test_mem_mpu.c`	11 MPU test cases
	`tests/memory/test_mem_common.c`	Host stubs + test main

Porting to a New Architecture

Only the arch files change:

1. Implement segment boundary setup and default protection
2. Implement `set_seg_perm()`, `unlock_nvm()`, `lock_nvm()` for your MPU

API at a Glance

Function	Returns	Complexity	Description
`tiku_mpu_init`	`void`	O(1)	Set boundaries + default protection
`tiku_mpu_set_permissions`	`void`	O(1)	Set permissions on one segment
`tiku_mpu_unlock_nvm`	`uint16_t`	O(1)	Enable writes, return saved state
`tiku_mpu_lock_nvm`	`void`	O(1)	Restore saved protection state
`tiku_mpu_scoped_write`	`void`	O(1)+fn	IRQ-safe unlock/write/lock
`tiku_mpu_enable_violation_nmi`	`void`	O(1)	Switch violations to NMI
`tiku_mpu_get_violation_flags`	`uint16_t`	O(1)	Read per-segment violation flags
`tiku_mpu_clear_violation_flags`	`void`	O(1)	Reset violation flags

Safety Guarantees

- NVM write-protected by default
- Hardware password prevents accidental unlock
- Scoped write disables interrupts
- Lock/unlock are idempotent

Design Choices

- Platform-independent kernel code
- All register access in arch layer
- HAL routes to correct platform
- Host-testable with stub registers
- Violation detection via latched flags

Test Configuration

Host-native (no hardware):

```
gcc -std=c99 -Wall -Wextra -I. \
  tests/memory/test_mem_common.c \
  tests/memory/test_mem_mpu.c \
  kernel/memory/tiku_mpu.c \
  -o test_tiku_mpu \
  && ./test_tiku_mpu
```

Host mode provides stub implementations of all HAL functions — plain variables mimic MPU registers so the test can inspect resulting state without hardware.

Host HAL stubs in test_mem_common.c

```c
static uint16_t stub_mpusam;

void tiku_mpu_arch_set_sam(uint16_t sam)
{
    stub_mpuctl0 = 0xA500;
    stub_mpusam  = sam;
    stub_mpuctl0 = 0xA500 | 0x0001;
}

void tiku_mpu_arch_set_default_protection(void
    )
{ tiku_mpu_arch_set_sam(0x0555); }

uint16_t tiku_mpu_arch_unlock_nvm(void) {
    uint16_t saved = stub_mpusam;
    tiku_mpu_arch_set_sam(saved | 0x0222);
    return saved;
}

/* Violation stub: check SAM bits */
void test_mpu_trigger_seg_violation(
    tiku_mpu_seg_t seg) { /* ... */ }
```

MPU Test Cases

#	Test Name	What It Verifies
21	Init defaults	SAM = 0x0555 (all R+X); CTL enable bit set
22	Unlock / Lock	Unlock returns saved state (0x0555); SAM becomes 0x0777; lock restores 0x0555
23	Set permissions	Changing one segment leaves others untouched
24	Scoped write	Callback invoked with write bits set; SAM relocked after return
25	Idempotency	Lock when locked is no-op; double unlock returns correct states
26	All segments	Set each segment independently; changing one preserves others
27	Permission flags	Each `TIKU_MPU_*` enum maps to correct SAM bits
28	Re-init restores	Custom permissions wiped by re-init to default 0x0555
29	Unlock custom base	Unlock with non-default base preserves R/X bits, adds W
30	Scoped write custom	Scoped write with custom base permissions works correctly
31	Violation detection	Write to locked segment sets flag; unlocked write does not

Test Deep Dive: Scoped Write

test_mem_mpu.c — Scoped write test

```c
typedef struct {
    int called;
    uint16_t sam_during;
} scoped_write_ctx_t;

static void scoped_write_cb(void *arg) {
    scoped_write_ctx_t *ctx = (scoped_write_ctx_t *)arg;
    ctx->called = 1;
    ctx->sam_during = tiku_mpu_arch_get_sam();  /* capture state during write */
}

void test_mpu_scoped_write(void) {
    scoped_write_ctx_t ctx = { 0, 0 };

    tiku_mpu_init();
    tiku_mpu_scoped_write(scoped_write_cb, &ctx);

    TEST_ASSERT(ctx.called == 1);                          /* callback was invoked */
    TEST_ASSERT((ctx.sam_during & 0x0222) == 0x0222);      /* write bits set during */
    TEST_ASSERT(tiku_mpu_arch_get_sam() == 0x0555);        /* locked after return */
}
```

What This Proves

Test Deep Dive: Violation Detection

test_mem_mpu.c — Violation detection test

```c
void test_mpu_violation_detect(void) {
    tiku_mpu_init();
    tiku_mpu_enable_violation_nmi();
    tiku_mpu_clear_violation_flags();

    TEST_ASSERT(tiku_mpu_get_violation_flags() == 0);    /* clean baseline */

    /* Phase 1: write while locked -> violation flag set */
    test_mpu_trigger_seg_violation(TIKU_MPU_SEG1);       /* host stub */
    TEST_ASSERT(tiku_mpu_get_violation_flags() != 0);    /* violation detected */

    tiku_mpu_clear_violation_flags();
    TEST_ASSERT(tiku_mpu_get_violation_flags() == 0);    /* flags cleared */

    /* Phase 2: write while unlocked -> no violation */
    saved = tiku_mpu_unlock_nvm();
    tiku_mpu_clear_violation_flags();
    test_mpu_trigger_seg_violation(TIKU_MPU_SEG1);
    TEST_ASSERT(tiku_mpu_get_violation_flags() == 0);    /* no violation */
    tiku_mpu_lock_nvm(saved);
}
```

Host vs Hardware

Summary

MPU Wrapper Design

- Hardware NVM write-protection
- Default: read+execute, no write
- Explicit unlock/lock pattern
- Scoped write with IRQ protection
- Per-segment fine-grained control
- Password-protected registers
- Violation detection via NMI
- Platform-independent kernel code
- All hardware access in arch layer

Test Coverage

- 11 MPU test cases
- Runs on host GCC with stub registers
- Covers: init, unlock/lock, per-segment, scoped write, idempotency, all flags, re-init, custom base, violation detection

Part of the Memory Module

The MPU wrappers extend `tiku_mem.h` alongside the arena allocator and persistent store — same header, same HAL pattern, same test binary.

Thank You

TikuOS MPU Write-Protection Wrappers

FRAM Safety for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
API Header:	`kernel/memory/tiku_mem.h`
MPU Logic:	`kernel/memory/tiku_mpu.c`
Tests:	`tests/memory/test_mem_mpu.c`